

May 4, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
```

```

import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "223"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\223

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen2.2\Code\256\Diagramme

Runden signifikanter Stellen

```

[3]: def round_to_sigs(val, errVal, einheiten):
    """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↪ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert
    """

```

```

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
    """

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳ signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳ hinzu
        exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *
↳ scale # mit scale wird das Komma so verschoben, dass alle
↳ signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳ weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳ * scale

qty = f"\qty{{{val_round}}({err_round})}{{{einheiten}}}"
num = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳ excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳ gewollt sind, die 10^e Prefixe beinhalten)

```

Gausssche Fehlerfortpflanzung

```
[4]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪ genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParameters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
        term = sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
    ↪ quadratisch aufsummiert
        errProneParameters.append((variable, Delta))

    absolute_err = sp.simplify(sp.sqrt(error), rational = True)
```

```

relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParameters

```

Sigma-Abweichungen

```

[5]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
    ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    abweichung_round = (abweichung / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_CEILING) * scale
    res = f"\\num{{{abweichung_round}}}"

    return float(abweichung_round),res

```

```

[6]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2

```

```

#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    VAL2=round_to_sigs(val2,err2,r'[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
    """)
    print(output)

```

```

[7]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

```

```

#Erstellung von Vergleichstabellen in Latex
def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val2)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        \\
        \\rightarrow{{nam1}}[\\unit{{{{einheiten}}}}]&{{nam2}}[\\unit{{{{einheiten}}}}]&\\text{{{{abs.Abw.
        \\rightarrow}}}[\\unit{{{{einheiten}}}}]&\\text{{{{proz.Abw.
        \\rightarrow}}}[\\unit{{{{percent}}}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&\\num{{{{val2}}}}&{{ABS}}&{{REL}}&{{SIGMA}}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{{nam1}}}}
        \\end{{table}}
        """)
    print(output)

```

```

[8]: Z=np.array([22,26,28,29,30,40,42,47])
#K_alpha (Ti, Fe, Ni, Cu, Zn, Zr, Mo, Ag) in keV:
K_alpha=np.array([4.61,6.44,7.52,8.10,8.72,15.84,17.49,21.84])
Delta_K_alpha=np.array([0.17,0.18,0.19,0.19,0.19,0.18,0.19,0.19])
sqrt_K_alpha=np.sqrt(K_alpha)
Delta_sqrt_K_alpha=Delta_K_alpha/2/np.sqrt(K_alpha)

n1=1
n2=2
def fit_func(x, sqrt_ER, sig12):
    return sqrt_ER*(x-sig12)*np.sqrt(1/n1**2-1/n2**2)
popt, pcov=curve_fit(fit_func, Z,sqrt_K_alpha,sigma=Delta_sqrt_K_alpha)
E_R,Delta_E_R,_,latexer=round_to_sigs(popt[0]**2,2*popt[0]*np.
    \\rightarrow sqrt(pcov[0][0]),r'\\kilo\\electronvolt')
sigma,Delta_sigma,_,latexsigma=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'[1]

```

```

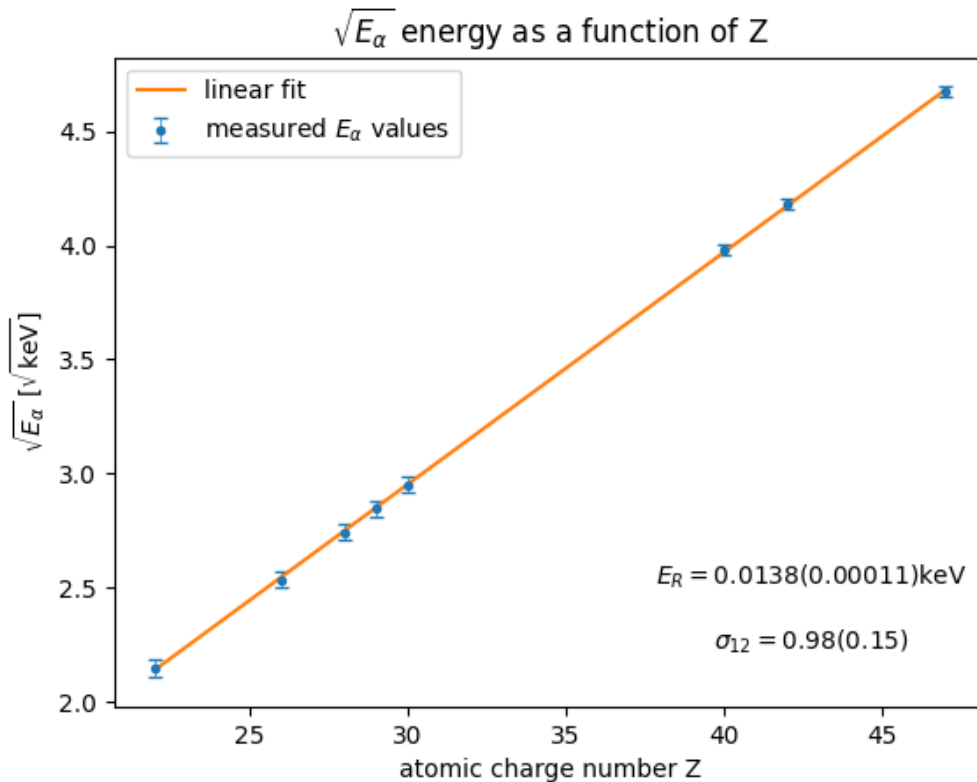
plt.close('all')
plt.errorbar(Z, sqrt_K_alpha, Delta_sqrt_K_alpha, fmt=".",
    ↪", capsize=3, elinewidth=0.5, label=r'measured $E_{\alpha}$ values',)
plt.plot(Z, fit_func(Z,*popt),label='linear fit')
plt.xlabel(r'atomic charge number Z')
plt.ylabel(r'$\sqrt{E_{\alpha}}$ [$\sqrt{\mathrm{keV}}$]')
plt.title(r'$\sqrt{E_{\alpha}}$ energy as a function of Z')
plt.text(0.8, 0.2, rf'$E_R={E_R}({\Delta E_R})$keV',
    ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
    ↪transAxes)
plt.text(0.8, 0.1, rf'$\sigma_{12}={\sigma}({\Delta \sigma})$',
    ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
    ↪transAxes)
print(latexer)
print(E_R)
print(latexsigma)
plt.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Kalpha.png",format="png")

```

$\sqrt{0.01380(0.00011)}$ $\sqrt{\text{kilo}\text{electronvolt}}$

0.0138

$\sqrt{0.98(0.15)}$




```
[9]: # compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
compare_table_withliterature(r'E_R',r'E_{R,\text{lit}}',r'\electronvolt',13.8,0.
↪11,13.605141380)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
E_R[\unit{\electronvolt}]&E_{R,\text{lit}}[\unit{\electronvolt}]&\text{abs. Abw.}[\unit{\electronvolt}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{\num{13.80(0.11)}}&\text{\num{13.60514138}}&\text{\num{0.19(0.11)}}&\text{\num{1.4(0.9)}}&\text{\num{1.8}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_E_R}
\end{table}
```

```
[10]: compare_table_withliterature(r'\sigma_{12}',r'\sigma_{12,\text{lit}}',r'',0.
↪9751,0.15,1.0)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
\sigma_{12}[\unit{ }]&\sigma_{12,\text{lit}}[\unit{ }]&\text{abs. Abw.}[\unit{ }]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{\num{0.98(0.15)}}&\text{\num{1.0}}&\text{\num{0.02(0.15)}}&\text{\num{2(16)}}&\text{\num{0.17}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_\sigma_{12}}
\end{table}
```

```
[11]: #K_beta (Fe, Ni, Cu, Zn, Zr, Mo, Ag) in keV:
K_beta=np.array([7.04,8.28,8.97,9.66,17.69,19.56,24.49])
Delta_K_beta=np.array([0.32,0.27,0.25,0.19,0.18,0.16,0.23])
sqrt_K_beta=np.sqrt(K_beta)
Delta_sqrt_K_beta=Delta_K_beta/2/np.sqrt(K_beta)

n1=1
```

```

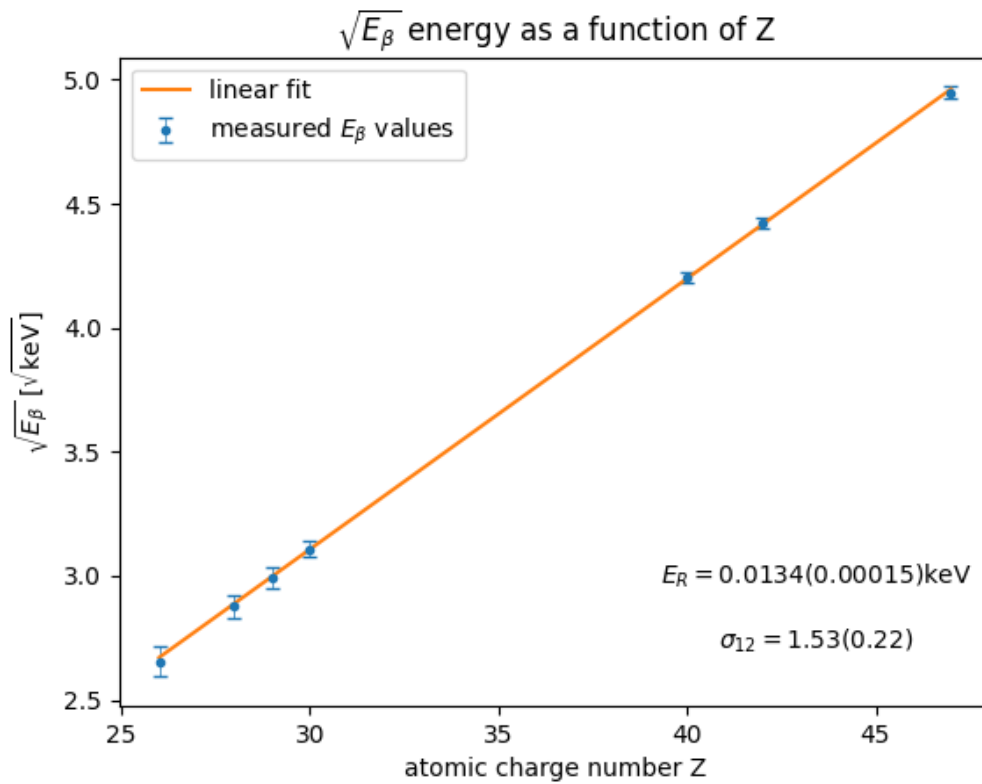
n2=3
def fit_func(x, sqrt_ER, sig12):
    return sqrt_ER*(x-sig12)*np.sqrt(1/n1**2-1/n2**2)
popt, pcov=curve_fit(fit_func,Z[1:],sqrt_K_beta,sigma=Delta_sqrt_K_beta)
E_R,Delta_E_R,_,latexer=round_to_sigs(popt[0]**2,2*popt[0]*np.
    ↳sqrt(pcov[0][0]),r'\kilo\electronvolt')
sigma,Delta_sigma,_,latexsigma=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'')

plt.close('all')
plt.errorbar(Z[1:], sqrt_K_beta, Delta_sqrt_K_beta, fmt=".",
    ↳",capsize=3,elinewidth=0.5,label=r'measured $E\_beta$ values',)
plt.plot(Z[1:], fit_func(Z[1:],*popt),label='linear fit')
plt.xlabel(r'atomic charge number Z')
plt.ylabel(r'$\sqrt{E\_beta}$ [$\sqrt{\mathrm{keV}}$]')
plt.title(r'$\sqrt{E\_beta}$ energy as a function of Z')
plt.text(0.8, 0.2,rf'$E_R={E_R}({Delta_E_R})$keV',␣
    ↳horizontalalignment='center',verticalalignment='center', transform=plt.gca().
    ↳transAxes)
plt.text(0.8, 0.1,rf'$\sigma_{{12}}={sigma}({Delta_sigma})$',␣
    ↳horizontalalignment='center',verticalalignment='center', transform=plt.gca().
    ↳transAxes)
print(latexer)
print(latexsigma)
plt.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Kbeta.png",format="png")

```

\qty{0.01340(0.00015)}{\kilo\electronvolt}

\qty{1.53(0.22)}{}



```
[12]: compare_table_withliterature(r'E_R',r'E_{R,\text{lit}}',r'\electronvolt',13.
    ↪40,0.15,13.605141380)

compare_table_withliterature(r'\sigma_{13}',r'\sigma_{12,\text{lit}}',r'',1.
    ↪53,0.22,1.8)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
E_R[\unit{\electronvolt}]&E_{R,\text{lit}}[\unit{\electronvolt}]&\text{abs. Abw.}[\unit{\electronvolt}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\midrule
\num{13.40(0.15)}&\num{13.60514138}&\num{0.21(0.15)}&\num{1.5(1.2)}&\num{1.4}\midrule
\bottomrule
\end{tabularx}
\label{table:Vergleich_E_R}
\end{table}
```

```

\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
\sigma_{13}[\unit{ }]&\sigma_{12,\text{lit}}[\unit{ }]&\text{abs.Abw.}[\unit{ }]&\text{proz.Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\num{1.53(0.22)}&\num{1.8}&\num{0.27(0.22)}&\num{15(13)}&\num{1.3}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_\sigma_{13}}
\end{table}

```

```

[13]: data1="""
# M 416 921.5
# L 416 860
# L 361.5 860
# L 307 860
# L 307 849.49
# L 307 838.99
# L 371.25 839.24
# L 435.5 839.5
# L 435.76 911.25
# L 436.01 983
# L 426.01 983
# L 416 983
# Z

"""
data2="""
M 485 974.98
C 476.49 971.01 471.39 964.09 470.32 955.06
C 469.11 944.83 469.08 944.88 517.45 889.81
C 541.88 862 563.87 837.61 566.31 835.61
C 568.75 833.6 584.42 824.51 601.12 815.39
C 617.83 806.28 632.29 798.12 633.25 797.26
C 635.34 795.39 635.52 792.67 633.65 791.12
C 632.87 790.48 608.39 787.86 575.75 784.93
L 519.19 779.86
L 461.42 804.6
C 408.88 827.1 403.15 829.33 398.08 829.32
C 376.59 829.28 366.09 802.58 381.73 787.74
C 383.64 785.92 403.85 776.69 440.57 760.87
C 471.33 747.61 498.98 735.89 502 734.82
C 512.93 730.95 514.47 730.97 578.5 735.92

```

```

C 612.05 738.51 639.53 740.6 639.57 740.57
C 640.05 740.14 693.03 612.71 692.81 612.52
C 692.64 612.38 678.55 602.04 661.5 589.55
C 644.45 577.05 629.22 565.63 627.65 564.16
C 626.09 562.7 623.34 558.58 621.54 555
C 613.39 538.77 576.16 457.19 575.71 454.57
C 574.39 446.92 580.25 436.58 587.72 433.38
C 595.65 429.98 607.05 433.09 611.52 439.87
C 612.84 441.87 623.4 463.95 635 488.95
C 646.6 513.95 656.62 535.04 657.26 535.82
C 658.71 537.56 709.84 574.02 719.5 580.19
C 723.35 582.65 738.65 591.03 753.5 598.82
L 780.5 612.97
L 821.21 646.26
C 843.6 664.58 863.54 681.64 865.52 684.19
C 868.89 688.51 870.11 692.46 883.6 742.66
C 899.09 800.27 899.05 800.06 894.95 808.09
C 888.77 820.21 868.91 821.37 861.42 810.05
C 859.7 807.46 855.02 791.54 845.81 756.92
L 832.65 707.5
L 806.75 686.15
C 792.51 674.41 780.71 664.96 780.52 665.15
C 780.34 665.34 767.57 695.59 752.14 732.37
C 736.72 769.15 722.9 800.98 721.44 803.09
C 719.97 805.21 717.16 808.16 715.2 809.66
C 713.23 811.16 686.62 825.54 656.06 841.61
C 625.5 857.68 598.83 872.24 596.78 873.95
C 594.73 875.66 574.35 898.36 551.48 924.38
C 523.23 956.51 508.6 972.42 505.84 973.97
C 499.96 977.27 490.86 977.72 485 974.98
Z
"""
data3="""
M 764.27 588.1
C 751.75 583.46 742.28 573.67 737.96 560.92
C 736.22 555.76 735.87 552.83 736.19 545.98
C 736.93 530.25 745.43 517.36 759.72 510.29
C 765.99 507.2 768.39 506.57 775.4 506.2
C 789.36 505.47 799.42 509.47 808.93 519.52
C 828.35 540.06 822.03 573.5 796.34 586.15
C 790.29 589.13 788.61 589.49 779.56 589.75
C 771.56 589.98 768.43 589.64 764.27 588.1
Z
"""
data = [data1, data2, data3]

```

```

pattern = r'([MLCZ])\s*([\d\s.-]+)'      # ([]) ist Gruppe, speichert in
↳ Gruppe 1 den Buchstaben [M oder L oder C] ab      \s* beliebig viele
↳ Leerzeichen  () Gruppe zwei

plt.close('all')

for i in range(3):
    x=[]
    y=[]
    for command, values in re.findall(pattern, data[i]):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?[d*]\.?[d+]', values)] #[-+]??:
        ↳ negative Zahlen      \d*: beliebig viele Ziffern      \.?: optionaler
        ↳ Dezimalpunkt      \d+: Sucht nach mindestens einer Ziffer nach dem Punkt und
        ↳ wird von leerzeichen gestoppt

        if command == 'M' or command == 'L':
            # Nimm das erste (und einzige) Paar: [x, y]
            x.append(nums[0])
            y.append(nums[1])

        elif command == 'C':
            # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
            # Wir wollen nur das letzte Paar (Index 4 und 5)
            x.append(nums[4])
            y.append(nums[5])

        elif command == 'Z':
            x.append(x[0])      # Startpunkt = Endpunkt
            y.append(y[0])

    x=np.array(x)
    y=1123-np.array(y)
    if i == 0:
        # Nur beim ersten Durchgang das Label setzen
        plt.plot(x, y, marker='.', color='red', linewidth=1, label='particle
↳ position')
    else:
        # Danach ohne Label plotten
        plt.plot(x, y, marker='.', color='red', linewidth=1)
# plt.title('Deterministisches Leiden eines Studierenden-Partikels')
plt.xlabel(r'$x\,, [\mu m]$')
plt.ylabel(r'$y\,, [\mu m]$')
plt.legend(loc='lower right')

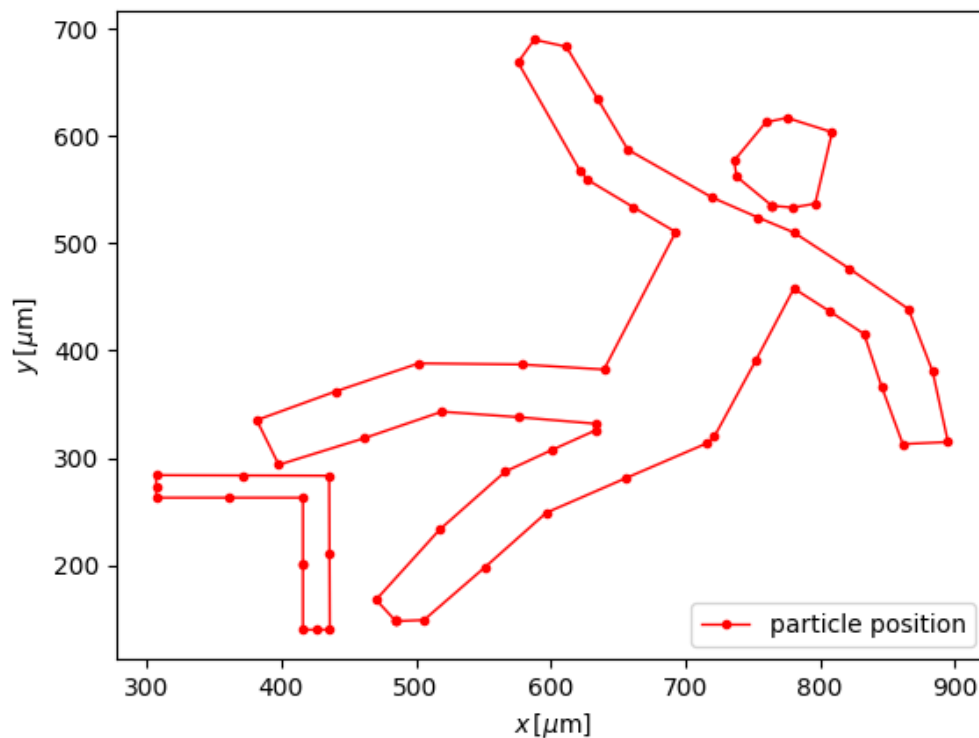
plt.show()

```

```
plt.savefig(Ausgabe_path/"Meme.png",format="png")
```

```
# pattern = r'\d*\.\d*'
# coordinates=re.findall(pattern, data)
# x=[]
# y=[]
# for i in range(0, len(coordinates), 2):
#     x.append(coordinates[i])
#     y.append(coordinates[i+1])

# for line in data.strip().splitlines():
#     matches = re.findall(pattern, line)
#     if len(matches) == 2:
#         xvalue = map(float, matches[0])
#         yvalue = map(float, matches[1])
#         x.append(xvalue)
#         y.append(yvalue)
```



```

[19]: data=""
M 14.19,835.41
C 7.94,832.29 3.59,827.28 1.38,820.64 -0.51,814.95 -0.05,806.94 2.44,802.12 3.
  ↪32,800.41 106.42,621.6 231.55,404.76 406.89,100.91 459.98,9.66 463.07,6.83
  ↪472.79,-2.07 487.21,-2.07 496.94,6.83 500.03,9.66 553.06,100.82 728.51,404.
  ↪86 853.66,621.76 956.76,800.58 957.62,802.23 960.06,806.95 960.5,814.99 958.
  ↪62,820.64 956.41,827.28 952.06,832.29 945.81,835.41
L 940.61,838
H 480 19.39
Z

M 888.83,786.03
C 888.14,783.99 480.48,78.05 480,78.06 479.53,78.06 71.88,784 71.17,786.05 70.
  ↪96,786.65 220.55,787 480,787 745.21,787 889.05,786.66 888.83,786.03
Z

M 459.14,763.97
C 438.41,761.96 415.79,756.9 398,750.29 386.1,745.87 373,739.95 373,738.99
  ↪373,738.14 444.62,613.98 445.78,612.82 446.21,612.39 449.71,613.32 453.
  ↪57,614.89 463.16,618.8 469.49,620 480.3,619.99 490.65,619.97 499.03,618.29
  ↪507.56,614.52 510.79,613.09 513.73,612.33 514.21,612.81 515.43,614.03
  ↪587,738.07 587,738.97 587,739.37 583.96,741.09 580.25,742.79 558.3,752.82
  ↪537.23,758.96 512.5,762.55 499.8,764.39 471.32,765.15 459.14,763.97
Z

M 468.63,600.41
C 451.75,596.33 436.88,581.42 433.02,564.72 428.54,545.28 437.25,523.75 453.
  ↪93,513.05 462.22,507.73 469.37,505.72 480,505.72 487.21,505.72 490.93,506.23
  ↪495.42,507.86 515.32,515.08 528,532.86 527.96,553.5 527.94,566.9 524.09,576.
  ↪63 514.92,586.43 502.87,599.31 485.61,604.52 468.63,600.41
Z

M 265.53,548.75
C 269.51,513.61 275.97,490.4 289.55,462.5 303.35,434.15 325.23,405.73 348.
  ↪28,386.2 360.54,375.82 372.16,367.76 373.27,368.87 373.78,369.38 390.54,398.
  ↪08 410.5,432.64
L 446.78,495.47 442.13,498.6
C 435.51,503.06 425.73,513.86 421.66,521.21 417.4,528.9 414.45,538.32 413.
  ↪49,547.27
L 412.77,554
H 338.85 264.93
Z

M 546.51,547.27

```



```

C 545.55,538.32 542.6,528.9 538.34,521.21 534.39,514.07 524.7,503.26 518.3,498.
↳87 515.94,497.24 514,495.5 514,495.01 514,493.93 584.79,371.04 586.46,369.21
↳588.96,366.49 617.14,389.37 632.82,406.86 668.71,446.89 688.81,493.22 694.
↳38,548.75
L 694.91,554
H 621.07 547.23
Z
"""
data_blocks = [b.strip() for b in data.split('\n\n') if b.strip()]

pattern = r'([MLCZHV])s*([\d\s.,-]+)?'      # ([]) ist Gruppe, speichert in
↳Gruppe1 den Buchstaben [M oder L oder C] ab    \s* beliebig viele
↳Leerzeichen  ()Gruppe zwei

plt.close('all')

for i, block in enumerate(data_blocks):
    x=[]
    y=[]
    for command, values in re.findall(pattern, block):
        # Alle Zahlen im Block extrahieren und in Floats umwandeln
        nums = [float(n) for n in re.findall(r'[-+]?[d*\.]?[d+]', values.
↳replace(',', ' '))] #[-+]??: negative Zahlen        \d*: beliebig viele
↳Ziffern          \.?: optionaler Dezimalpunkt      \d+: Sucht nach mindestens
↳einer Ziffer nach dem Punkt und wird von Leerzeichen gestoppt

        # elif command == 'C' or command == 'c':
        #     # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
        #     # Wir wollen nur das letzte Paar (Index 4 und 5)
        #     x.append(nums[-2])
        #     y.append(nums[-1])

    if command == 'M' or command == 'L':
        if len(nums) >= 2:
            last_x, last_y = nums[0], nums[1]
            x.append(last_x)
            y.append(last_y)

    elif command == 'C':
        # Wir gehen in 6er-Schritten durch die Zahlen (i = 0, 6, 12...)
        for j in range(0, len(nums), 2):
            # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
            # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
            x.append(nums[j])
            y.append(nums[j+1])

        # for j in range(0, len(nums), 6):

```

```

        # # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
        #     if j + 5 < len(nums):
        #         # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
        #         x.append(nums[j+4])
        #         y.append(nums[j+5])
    elif command == 'H':
        # Nur X ändert sich, Y bleibt gleich
        if len(nums) >= 1:
            last_x = nums[0]
            x.append(last_x)
            y.append(last_y)

    elif command == 'V':
        # Nur Y ändert sich, X bleibt gleich
        if len(nums) >= 1:
            last_y = nums[0]
            x.append(last_x)
            y.append(last_y)

    elif command == 'Z':
        if x:
            x.append(x[0])
            y.append(y[0])
            last_x, last_y = x[0], y[0]

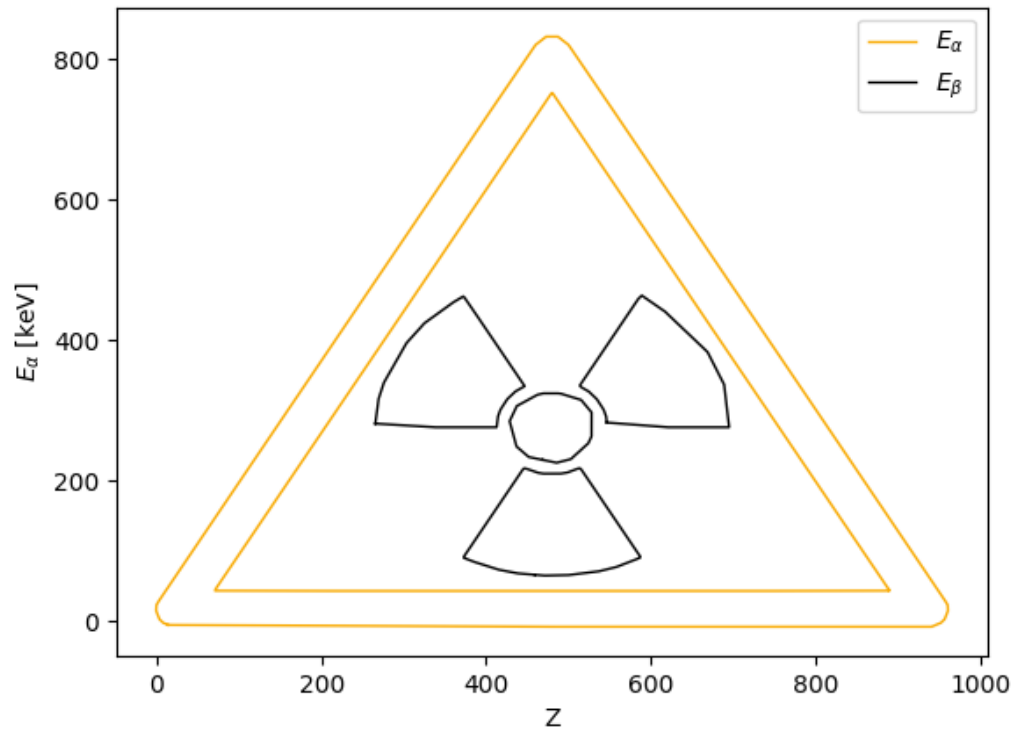
x=np.array(x)
y=830-np.array(y)
if i == 0:
    labeltext=r'$E\_alpha$'
elif i==2:
    labeltext=r'$E\_beta$'          # Nur beim nullten Durchgang das Label
↪setzen
else: labeltext=None

if i>1:
    plt.plot(x, y,color=f"black",linewidth=1, label=labeltext)#color=f"C{1}"
else:
    plt.plot(x, y,color='#f9aa00',linewidth=1,
↪label=labeltext)#color=f"C{1}"

plt.xlabel(r'Z')
plt.ylabel(r'$E\_alpha$ [keV]')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Meme.png",format="png")

```



```
[18]: data=""
M 14.19,835.41
C 7.94,832.29 3.59,827.28 1.38,820.64 -0.51,814.95 -0.05,806.94 2.44,802.12 3.
↪32,800.41 106.42,621.6 231.55,404.76 406.89,100.91 459.98,9.66 463.07,6.83
↪472.79,-2.07 487.21,-2.07 496.94,6.83 500.03,9.66 553.06,100.82 728.51,404.
↪86 853.66,621.76 956.76,800.58 957.62,802.23 960.06,806.95 960.5,814.99 958.
↪62,820.64 956.41,827.28 952.06,832.29 945.81,835.41
L 940.61,838
H 480 19.39
Z

M 888.83,786.03
C 888.14,783.99 480.48,78.05 480,78.06 479.53,78.06 71.88,784 71.17,786.05 70.
↪96,786.65 220.55,787 480,787 745.21,787 889.05,786.66 888.83,786.03
Z

M 459.14,763.97
```

```

C 438.41,761.96 415.79,756.9 398,750.29 386.1,745.87 373,739.95 373,738.99
↳373,738.14 444.62,613.98 445.78,612.82 446.21,612.39 449.71,613.32 453.
↳57,614.89 463.16,618.8 469.49,620 480.3,619.99 490.65,619.97 499.03,618.29
↳507.56,614.52 510.79,613.09 513.73,612.33 514.21,612.81 515.43,614.03
↳587,738.07 587,738.97 587,739.37 583.96,741.09 580.25,742.79 558.3,752.82
↳537.23,758.96 512.5,762.55 499.8,764.39 471.32,765.15 459.14,763.97
Z

M 468.63,600.41
C 451.75,596.33 436.88,581.42 433.02,564.72 428.54,545.28 437.25,523.75 453.
↳93,513.05 462.22,507.73 469.37,505.72 480,505.72 487.21,505.72 490.93,506.23
↳495.42,507.86 515.32,515.08 528,532.86 527.96,553.5 527.94,566.9 524.09,576.
↳63 514.92,586.43 502.87,599.31 485.61,604.52 468.63,600.41
Z

M 265.53,548.75
C 269.51,513.61 275.97,490.4 289.55,462.5 303.35,434.15 325.23,405.73 348.
↳28,386.2 360.54,375.82 372.16,367.76 373.27,368.87 373.78,369.38 390.54,398.
↳08 410.5,432.64
L 446.78,495.47 442.13,498.6
C 435.51,503.06 425.73,513.86 421.66,521.21 417.4,528.9 414.45,538.32 413.
↳49,547.27
L 412.77,554
H 338.85 264.93
Z

M 546.51,547.27
C 545.55,538.32 542.6,528.9 538.34,521.21 534.39,514.07 524.7,503.26 518.3,498.
↳87 515.94,497.24 514,495.5 514,495.01 514,493.93 584.79,371.04 586.46,369.21
↳588.96,366.49 617.14,389.37 632.82,406.86 668.71,446.89 688.81,493.22 694.
↳38,548.75
L 694.91,554
H 621.07 547.23
Z
"""
data_blocks = [b.strip() for b in data.split('\n\n') if b.strip()]

pattern = r'([MLCZHV])\s*([\d\s.,-]+)?' # ([ ]) ist Gruppe, speichert in
↳Gruppe1 den Buchstaben [M oder L oder C] ab \s* beliebig viele
↳Leerzeichen ()Gruppe zwei

plt.close('all')

for i, block in enumerate(data_blocks):
    x=[]
    y=[]

```

```

for command, values in re.findall(pattern, block):
    # Alle Zahlen im Block extrahieren und in Floats umwandeln
    nums = [float(n) for n in re.findall(r'[-+]?\\d*\\.?(\\d+)', values.
↪replace(',', ' '))] #[-+]?: negative Zahlen      \\d*: beliebig viele
↪Ziffern      \\.: optionaler Dezimalpunkt      \\d+: Sucht nach mindestens
↪einer Ziffer nach dem Punkt und wird von Leerzeichen gestoppt

    # elif command == 'C' or command == 'c':
    #     # Ein C-Befehl hat 6 Zahlen (x1, y1, x2, y2, x3, y3)
    #     # Wir wollen nur das letzte Paar (Index 4 und 5)
    #     x.append(nums[-2])
    #     y.append(nums[-1])

    if command == 'M' or command == 'L':
        if len(nums) >= 2:
            last_x, last_y = nums[0], nums[1]
            x.append(last_x)
            y.append(last_y)

    elif command == 'C':
        for j in range(0, len(nums), 2):
            last_x, last_y = nums[j], nums[j+1]
            x.append(last_x)
            y.append(last_y)

    # # Wir gehen in 6er-Schritten durch die Zahlen (i = 0, 6, 12...)
    #     for j in range(0, len(nums), 6):
    #         # Wir prüfen, ob wir genug Zahlen für ein komplettes Segment haben
    #         if j + 5 < len(nums):
    #             # Der echte Knoten ist immer das letzte Paar einer 6er-Gruppe
    #             x.append(nums[j+4])
    #             y.append(nums[j+5])

    elif command == 'H':
        # Nur X ändert sich, Y bleibt gleich
        if len(nums) >= 1:
            last_x = nums[0]
            x.append(last_x)
            y.append(last_y)

    elif command == 'V':
        # Nur Y ändert sich, X bleibt gleich
        if len(nums) >= 1:
            last_y = nums[0]
            x.append(last_x)
            y.append(last_y)

    elif command == 'Z':

```

```

        if x:
            x.append(x[0])
            y.append(y[0])
            last_x, last_y = x[0], y[0]

x=np.array(x)
y=830-np.array(y)

if i==0 or i>1:
    if i == 0:
        labeltext=r'$E_{\alpha}$'
    else: labeltext=None
    plt.plot(x, y,color=f"black",linewidth=2, label=labeltext)#color=f"C{1}"
    plt.fill(x, y,color=f"black",)#color=f"C{1}"
else:
    if i==1:
        labeltext=r'$E_{\beta}$'          # Nur beim nullten Durchgang das
↪Label setzen
        plt.plot(x, y,color='#f9aa00',linewidth=2,
↪label=labeltext)#color=f"C{1}"
        plt.fill(x, y, color='#f9aa00',)#color=f"C{1}"

plt.xlabel(r'Z')
plt.ylabel(r'$E_{\alpha}$ [keV]')
plt.legend()

plt.show()
plt.savefig(Ausgabe_path/"Memefilled.png",format="png")

```

